ECE 264 - Object-Oriented Software Development

Chapter 3

Control Statements and Program Development

Core Objectives

- Statements:
  - if…else; while, for
  - break, continue
- Learn problem-solving skills: understanding problem requirements, dividing problems into smaller pieces, developing algorithms to solve problems and implementing those algorithms in code.
- Develop algorithms through the process of top-down, stepwise refinement.
- Create compound conditions with the Boolean operators

Algorithms

- Any computing problem can be solved by executing a series of actions in a specific order.
- An algorithm is a procedure for solving a problem in terms of
  - The actions to execute and
  - The order in which these actions execute
- Specifying the order in which statements (actions) execute in a program is called program control.

Pseudocode

- An informal language that helps you develop algorithms without having to worry about the strict details of Python syntax.
- Similar to everyday English.
- Helps you “think out” a program before attempting to write it in a programming language.
- Carefully prepared pseudocode can easily be converted to a corresponding program.
- Pseudocode normally describes only statements representing the actions that occur (e.g., input, output or calculations) after you convert a program from pseudocode to Python and run the program.

Pseudocode

Prompt the user to enter the first integer
Input the first integer

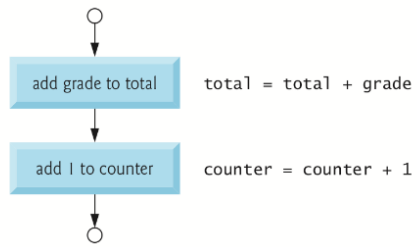
Prompt the user to enter the second integer
Input the second integer

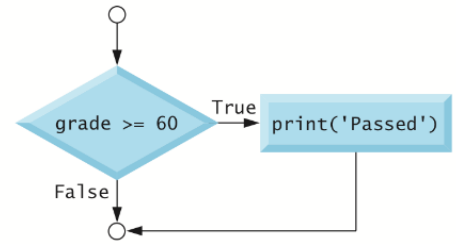
Add first integer and second integer, store their sum
Display the numbers and their sum

Control Statements

- Usually, statements in a program execute in the order in which they are written.
  - Called sequential execution.
- Various Python statements enable you to specify that the next statement to execute may be other than the next one in sequence.
  - Called transfer of control and is achieved with Python control statements.
- All programs can be written using three forms of control
  - Sequential execution
  - The selection statement
  - The iteration statement.
- Python statements execute one after the other “in sequence,” unless directed otherwise.

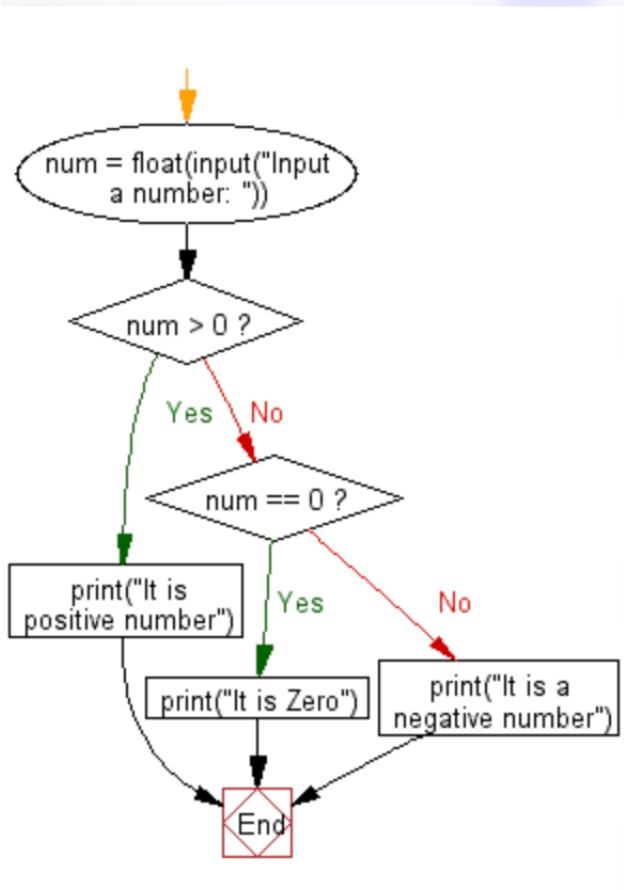
Flowcharts





- Graphical representation of an algorithm or a part of one.
- Drawn using rectangles, diamonds, rounded rectangles and small circles connected by arrows called flowlines.
- Clearly show how forms of control operate.
- **Rectangle** (or action) symbol indicates any action
- Diamond symbol to check conditions
- **Flowlines** show the order in which the actions execute.
- In a flowchart for a complete algorithm, the first symbol is a **rounded rectangle** containing the word "Begin." The last symbol is a **rounded rectangle** containing the word "End."
- In a flowchart for only a part of an algorithm, use small circles called connector symbols.

Flowchart Example



What's happening in this flow?

1. **Start**
2. **Input a number** → the program receives a value.
3. **Decision (Is number > 0?)**
   - Yes → print **"Positive"**
   - No → check the next condition.
4. **Decision (Is number == 0?)**
   - Yes → print **"Zero"**
   - No → print **"Negative"**
5. **End**

```python
num = float(input("Enter a number: "))

if num > 0:
    print("Positive")
elif num == 0:
    print("Zero")
else:
    print("Negative")
```

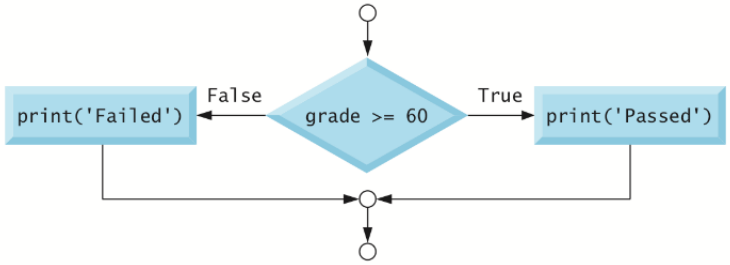
`if` Statement

- Three selection statements that execute code based on a condition – an expression that evaluates to either `True` or `False`:
  - `if` performs an action if a condition is `True` or skips the action if the condition is `False`.
  - `if…else` statement performs an action if a condition is `True` or performs a different action if the condition is `False`.
  - `if…elif…else` statement performs one of many different actions, depending on the truth or falsity of several conditions.
- Anywhere a single action can be placed, a group of actions can be placed.

Example

- `if…else`: Performs different suites, based on whether a condition is `True` or `False`.

If student's grade is greater than or equal to 60
    Display 'Passed'
Else
    Display 'Failed'



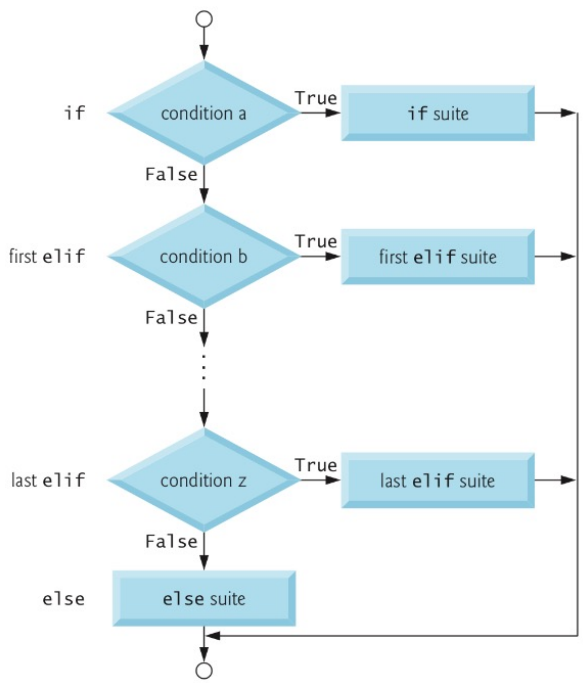
- Multiple Statements in a Suite

```
if grade >= 60:
    print('Passed')
else:
    print('Failed')
    print('You must take this course again')
```

```
if grade >= 60:
    print('Passed')
else:
    print('Failed')
print('You must take this course again')
```

Example

- `if…elif…else`: can test for many cases. Only the action for the ***first*** `True` condition executes.




```
grade = 77
```

```
if grade >= 90:
    print('A')
elif grade >= 80:
    print('B')
elif grade >= 70:
    print('C')
elif grade >= 60:
    print('D')
else:
    print('F')
```

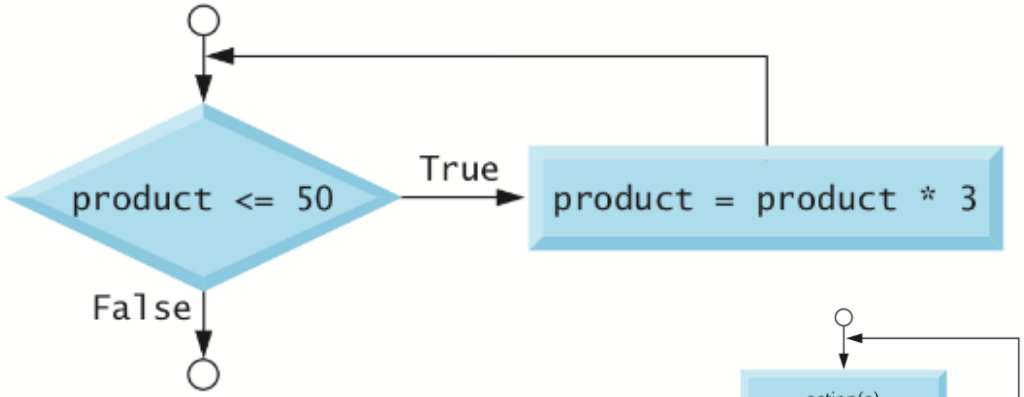
- `else` is optional.
- Handle values that do not satisfy any of the conditions
- Without an `else`, if no conditions are `True`, the program does not execute any of the statement's suites.

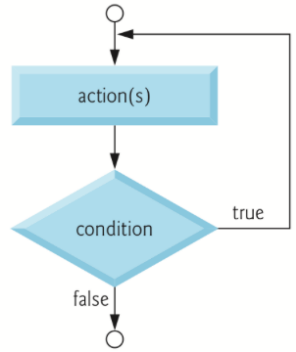
Iteration Statements

- Two iteration statements
  - `while` repeats an action (or a group of actions) as long as a condition remains `True`.
  - `for` statement repeats an action (or a group of actions) for every item in a sequence of items.

`while` Statement

- Repeats one or more actions while a condition remains `True`.






```
product = 3
```

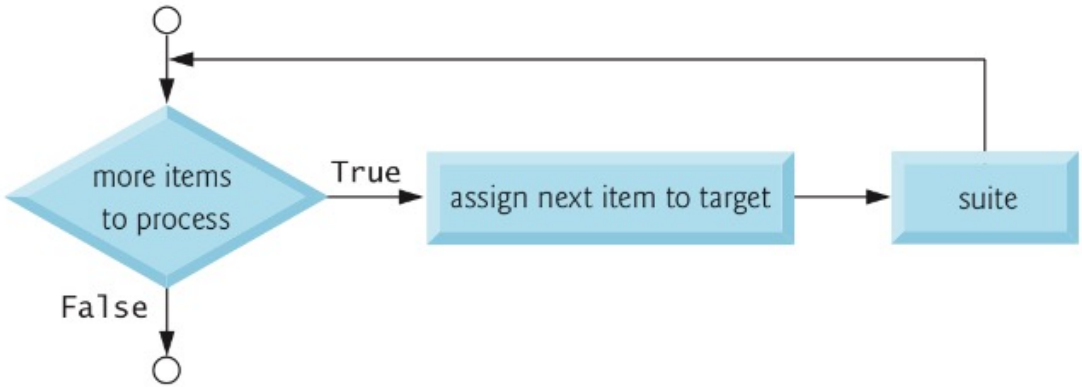
```
while product <= 50:
    product = product * 3
```

```
product
```

`for` Statement

- Repeat an action or several actions for each item in a sequence of items.
- A string is a sequence of individual characters.

```
for character in 'Programming':
    print(character, end='  ')
```



- `print` displays its argument(s), then moves the cursor to the next line.
- Can change this behavior with the argument `end`:
- `end` is a **keyword argument**, but it's not a Python keyword
- `sep` is another keyword argument

`for` Statement

```
total = 0
```

```
for number in [2, -3, 0, 17, 9]:
    total = total + number
```

```
total
```

```
for counter in range(10):
    print(counter, end=' ')
```

- The sequence to the right of the `for` statement's in keyword must be an iterable
- An object from which the `for` statement can take one item at a time.
- One of the most common iterables is a ***list***, which is a comma-separated collection of items enclosed in square brackets ([and]).
- `range` function: Creates an iterable object that represents a sequence of consecutive integer values starting from 0 and continuing up to, but not including, the argument value.

`break` & `continue` Statements

- Executing a break statement in a while or for immediately exits that statement.
- Executing a continue statement in a while or for loop skips the remainder of the loop's suite.
  - In a while, the condition is then tested to determine whether the loop should continue executing.
  - In a for, the loop processes the next item in the sequence (if any)

```
for number in range(100):
    if number == 10:
        break
    print(number, end=' ')
```

```
for number in range(10):
    if number == 5:
        continue
    print(number, end=' ')
```

Augmented Assignments

- Abbreviate assignment expressions in which the same variable name appears on the left and right of the =.
- `c=3, d=5, e=4, f=2, g=9, g=12`

Augmented assignment	Sample expression	Explanation	Assigns
`+=`	`c += 7`	`c = c + 7`	`10 to c`
`-=`	`d -= 4`	`d = d - 4`	`1 to d`
`*=`	`e *= 5`	`e = e * 5`	`20 to e`
`**=`	`f **= 3`	`f = f ** 3`	`8 to f`
`/=`	`g /= 2`	`g = g / 2`	`4.5 to g`
`//=`	`g //= 2`	`g = g // 2`	`4 to g`
`%=`	`h %= 9`	`h = h % 9`	`3 to h`

Boolean Operators

- `and`
- `or`
- `not`
- Precedence and grouping of the operators

Operators	Grouping
`()`	left to right
`**`	right to left
`*  /  //  %`	left to right
`+  -`	left to right
`<  <=  >  >=  ==  !=`	left to right
`not`	left to right
`and`	left to right
`or`	left to right

High Precedence

Low Precedence

Question: Assume that `i=1`, `j=2`, `k=3`, and `m=2`. What does each of the following conditions display?

a) `(i >= 1) and (j < 4)`
b) `(m <= 99) and (k < m)`
c) `(j >= i) or (k == m)`
d) `(k + m < j) or (3 - j >= k)`
e) `not (k > m)`

Program Development – Sequence-Controlled Iteration

- Most challenging part of solving a problem on a computer is developing an algorithm for the solution.
- Once a correct algorithm has been specified, creating a working Python program from the algorithm is typically straightforward.
- Steps:
  - Requirements Statement
  - Pseudocode for the Algorithm
  - Coding the Algorithm in Python
  - Execution and Testing
- ***Problem***: A class of ten students took a quiz. Their grades (integers in the range 0 – 100) are 98, 76, 71, 87, 83, 90, 57, 79, 82, 94. Determine the class average on the quiz.

Set total to zero
Set grade counter to zero
Set grades to a list of the ten grades

For each grade in the grades list:
    Add the grade to the total
    Add one to the grade counter

Set the class average to the total divided by the number of grades
Display the class average

```python
# fig03_01.py
"""Class average program with sequence-controlled repetition."""

# initialization phase
total = 0  # sum of grades
grade_counter = 0
grades = [98, 76, 71, 87, 83, 90, 57, 79, 82, 94]  # list of 10 grades

# processing phase
for grade in grades:
    total += grade  # add current grade to the running total
    grade_counter += 1  # indicate that one more grade was processed

# termination phase
average = total / grade_counter
print(f'Class average is {average}')
```

```
Class average is 81.7
```

- **Initialization** phase – creates the variables needed to process the grades and set these variables to appropriate initial values
- **Processing** phase – processes the grades, calculating the running total and counting the number of grades processed so far.
- **Termination** phase – calculates and displays the class average

Program Development – Sentinel-Controlled Iteration

- ***Generalized class-average Problem***: Develop a class-averaging program that processes an arbitrary number of grades each time the program executes.
  - Read the first value before reaching the `while` statement.
  - The value input determines whether the program's flow of control should enter the while's suite.
  - If the condition is `False`, the user entered -1, so the suite does not execute.
  - If the condition is `True`, the suite executes, adding the grade value to the `total` and incrementing the `grade_counter`.
  - Then we input another grade from the user and the while's condition is tested again.
  - `grade` is always input immediately before the program tests the while condition.
  - When the sentinel value is input, the loop terminates, and the program does not add –1 to the `total`.
  - After the loop terminates, the `if…else` statement executes.

```python
# fig03_02.py
"""Class average program with sentinel-controlled iteration."""

# initialization phase
total = 0  # sum of grades
grade_counter = 0  # number of grades entered

# processing phase
grade = int(input('Enter grade, -1 to end: '))  # get one grade

while grade != -1:
    total += grade
    grade_counter += 1
    grade = int(input('Enter grade, -1 to end: '))

# termination phase
if grade_counter != 0:
    average = total / grade_counter
    print(f'Class average is {average:.2f}')
else:
    print('No grades were entered')
```

Question: Sentinel-controlled repetition uses a counter variable to control the number of times a set of instructions executes.

Program Development – Nested Control Statements

A college offers a course that prepares students for the state licensing exam for real-estate brokers. Last year, several of the students who completed this course took the licensing examination. The college wants to know how well its students did on the exam. You have been asked to write a program to summarize the results. You have been given a list of these 10 students. Next to each name is written a 1 if the student passed the exam and a 2 if the student failed.

Your program should analyze the results of the exam as follows:

1. *Input each test result (i.e., a 1 or a 2). Display the message "Enter result" each time the program requests another test result.*
2. *Count the number of test results of each type.*
3. *Display a summary of the test results indicating the number of students who passed and the number of students who failed.*
4. *If more than eight students passed the exam, display "Bonus to instructor."*

Pseudocode

Analyze exam results and decide whether instructor should receive a bonus

Initialize variables
Input the ten exam grades and count passes and failures
Summarize the exam results and decide whether instructor should receive a bonus

Initialize passes to zero
Initialize failures to zero

For each of the ten students
    Input the next exam result

    If the student passed
        Add one to passes
    Else
        Add one to failures

Display the number of passes
Display the number of failures

If more than eight students passed
    Display "Bonus to instructor"

```python
# fig03_03.py
"""Using nested control statements to analyze examination results."""

# initialize variables
passes = 0  # number of passes
failures = 0  # number of failures

# process 10 students
for student in range(10):
    # get one exam result
    result = int(input('Enter result (1=pass, 2=fail): '))

    if result == 1:
        passes = passes + 1
    else:
        failures = failures + 1

# termination phase
print('Passed:', passes)
print('Failed:', failures)

if passes > 8:
    print('Bonus to instructor')
```

Q1- What if the value entered is other than 1 or 2?

- Keep looping until the user enters a correct value.
- Use one counter to keep track of the number of passes, then calculate the number of failures after all the user's inputs have been received.

Q2. What is wrong with the following code?

```
a = b = 7
print('a =', a, '\nb =', b)
```

Q3 – What does the following program print?

```
for row in range(10):
    for column in range(10):
        print('<' if row % 2 == 1 else '>', end='')
    print()
```

```
>>>>>>>>>>
<<<<<<<<<<
>>>>>>>>>>
<<<<<<<<<<
>>>>>>>>>>
<<<<<<<<<<
>>>>>>>>>>
<<<<<<<<<<
>>>>>>>>>>
<<<<<<<<<<
```

Q4 – Fill in the missing code below.

- Replace the *** so that when you execute the code, it displays two rows, each containing seven @ symbols as in:

```
@@@@@@@
@@@@@@@
```

```
for ***:
    for ***:
        print(‘@’, end=‘’)
    print()
```

```
for row in range(2):
    for column in range(7):
        print('@', end='')
    print()
```

Q5 – Calculate change using fewest number of coins.

- Write a script that inputs a purchase price of a dollar or less for an item. Assume the purchaser pays with a dollar bill. Determine the amount of change the cashier should give back to the purchaser. Display the change using the ***fewest*** number of pennies, nickels, dimes and quarters.
- For example, if the purchaser is due 73 cents in change, the script would output:

```
Your change is:
2 quarters
2 dimes
3 pennies
```